

G05 / PROTECTED ANSWERS

G05 REVIEW B

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Architecture answer 3

Key answer 3

Group answer 3

Delivery answer 3

Schema answer 3

Spark answer 4

Lag answer 4

Teach-back 4

G05 REVIEW B - IMPLEMENTATION RUBRIC 4

Protected runtime rubric B 4

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Protected review. Read only after a saved gate attempt. Repair the weak concept, close this file, then take the fresh changed gate.

Architecture answer

Kafka carries retained partition logs; Spark or another processor can transform the stream; Delta provides a durable transactional analytical table; a real-time notification service is an independent consumer application. Keep these responsibilities separate.

Validation: name the source of truth for replay, the consumer group identities, the lakehouse table keys/grain and the notification idempotency boundary. An answer that says Kafka itself is the analytical table or that Delta pushes notifications confuses layers.

Key answer

Use `product_id` when each product price-change sequence must stay ordered. Using `seller_id` would impose an unnecessarily broad seller-wide ordering boundary and can hotspot large sellers.

Validation: show that all price changes for SKU1 share one routing key and partition sequence while other products can distribute. Seller totals can be computed downstream without forcing all seller products into one partition.

Group answer

Analytics and notification services need different group IDs because both must independently see every event. The six notification replicas share one notification group and divide that group partitions. Useful simultaneous parallelism is bounded by available partitions.

Validation: draw two logical groups. If analytics accidentally joins the notification group, records would be split between applications rather than independently delivered, which violates the requirement.

Delivery answer

Use stable `event_id` or product/version uniqueness at the sink so replay updates/returns the existing business effect instead of applying a second price change. Commit consumer progress only after a durable handled outcome.

Crash-window proof: write the price successfully, fail before progress commit, replay, and show the second attempt is idempotent. This is application/sink correctness; producer idempotence alone is not sufficient.

Schema answer

For v2, add currency as an optional/defaulted field under a compatible schema policy while old consumers still understand order/product and price. A destructive rename or type change can become an intentionally breaking v3 and requires coordinated consumer migration.

Validation: test old-reader/new-writer and new-reader/old-writer expectations according to the actual schema format/policy. Do not claim universal compatibility from a JSON example alone.

Spark answer

For a brand-new query with no usable checkpoint, `startingOffsets` helps choose the starting position. An existing valid checkpoint normally determines resumed source progress. Deleting the checkpoint is therefore a recovery action, not routine cleanup.

Validation: run a disposable query, save checkpoint/source offset evidence, restart with the same checkpoint and explain what resumed. For a corrective replay, use an intentionally new checkpoint and isolated target, then reconcile before promotion.

Lag answer

All-partition lag plus frequent rebalances after deployment points to broad processing/poll/session/resource or downstream regression rather than one hot key. Check the deployment delta, processing duration, poll/heartbeat/rebalance logs, broker health and sink latency/errors before scaling blindly.

Validation: show lag trend and rebalance frequency before/after the correction, plus output reconciliation. If lag falls only because records are skipped/quarantined incorrectly, the incident is not resolved.

Teach-back

Exactly once is not a magic global property. Name the source boundary, producer/transaction boundary, processor state/checkpoint, and external sink/idempotency mechanism. State where replay may occur and how it becomes harmless.

Validation: give one counterexample where Kafka-side exactly-once is intact but an email/payment repeats. If you cannot construct that counterexample, your boundary explanation is still too vague.

G05 REVIEW B - IMPLEMENTATION RUBRIC

Protected runtime rubric B

PASS implementation requires: independent B output/state; P2 duplicate one business effect; P3 schema issue explicit; valid sink failure remains uncommitted/recoverable; recovery + changed retry succeed; input/output/quarantine/business state reconcile.

Conceptual pass requires correct separate consumer groups, product key/order boundary, delivery semantics, compatible schema evolution reasoning, checkpoint/`startingOffsets` boundary, and all-partition lag diagnosis. Record implementation and reasoning scores separately.